

INVENTORY ATLAS

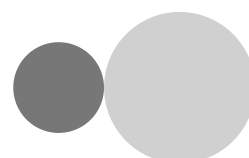
Дизайн-документ системи особистої інвентаризації

Від предмета до точного місця зберігання: каталог, вкладені коробки, фото, пошук, етикетки, сканування, доставка, експорт та майбутні продажі.

Версія документа: 0.3.1

Дата: 27 серпня 2026

Статус: final approved architecture specification after E1 errata



Керування документом

Поле	Значення
Робоча назва	Inventory Atlas
Тип документа	Product requirements + software design document
Цільова версія	v0.3.1 - фінальна основа для schema v1 та implementation blueprint
Головна аудиторія	Власник продукту, розробник, майбутні контриб'ютори
Статус рішень	Addendum A1-A3/B1-B11 та errata E1 закриті; заморожені рішення не змінені

Зміни v0.3.1

Пункт	Зміна
E1	Одна business transaction використовує один client; item_search/outbox мають trx-aware write ports
C1-C3	Уніфіковано ADR IDs; статусну схему позначено як приклад; connection budget віддано на load-test validation
A1	Data-access boundary та table ownership/client matrix зафіксовані як база E1
A2	Повний реєстр item_search invalidators; sync path/visibility і async tsvector
A3	Visibility x search matrix; private values не індексуються; unlisted policy
B1-B5	Розділено item/index timestamps; ltree label; i18n entities; locale labels; tree_root_id
B6-B10	Ізольований HEIC decode; bounded stale; OpenAPI -> Zod; bulk conflicts; distribution note
B11	MVP estimate скориговано до 22-30 тижнів для одного full-time розробника

Як читати документ

Розділи 1-4 пояснюють продукт і межі MVP. Розділи 5-11 є технічною специфікацією. Розділи 12-16 задають якість, безпеку, розгортання, етапи й критерії готовності.

Зміст

Керування документом	2
Зміни v0.3.1	2
Зміст	3
1. Резюме продукту	5
1.1 Проблема	5
1.2 Бачення	5
1.3 Цілі першого релізу	5
1.4 Нецілі MVP	5
2. Користувачі, ролі та доступ	6
2.1 Основні персони	6
2.2 Рольова модель	6
2.3 Політика публічності	6
3. Інформаційна модель	6
3.1 Предмет	6
3.2 Дерево зберігання	7
3.3 Правила дерева	7
3.4 Динамічні поля	8
3.5 Генератор display name	8
3.6 Статуси без змішування контекстів	8
4. Функціональні вимоги	8
4.1 Каталог і картка предмета	8
4.2 Додавання та редагування	9
4.3 Пошук і фільтри	9
4.4 Фото і медіа	9
4.5 Етикетки, QR і штрих-коди	9
4.6 Режим інвентаризації	9
4.7 Імпорт, експорт і backup	10
4.8 Доставка	10
4.9 Продажі	10
5. UX та інформаційна архітектура	10
5.1 Основна навігація	10
5.2 Mobile-first, але не mobile-only	10
5.3 Ключові сценарії	11
5.4 Дизайн-система	11
5.5 Локалізація	11
6. Технічна архітектура	12
6.1 Рекомендований стек	12
6.1.1 Власність таблиць і транзакційна межа	13
6.2 Чому TypeScript на backend	13
6.2.1 Контрактна безпека JavaScript frontend	13
6.3 Модульний моноліт	13
6.4 SOLID, OOP, KISS і DRY на практиці	14
7. Модель даних	14
7.1 Основні сутності	14
7.2 Представлення динамічних значень	15
7.3 Ідентифікатори	15

7.4 Цілісність	15
7.5 item_search як read model	15
7.5.1 Visibility x search policy	16
8. API design	16
8.1 Принципи	16
8.2 Основні endpoints	16
8.3 Події й інтеграції	17
9. Пошук, фільтрація та масштабування	17
9.1 Пошуковий документ	17
9.2 Індeksi	18
9.2.1 SQL policy	18
9.3 Пороги еволюції	18
10. Авторизація, безпека та аудит	18
10.1 Authentication	18
10.2 Обов'язкові контрзаходи	18
10.3 Audit	19
10.4 Threat model highlights	19
11. Інтеграції	19
11.1 Служби доставки	19
11.2 Майданчики продажу	19
11.3 Правила інтеграцій	19
12. Розгортання й експлуатація	20
12.1 Базовий Docker Compose	20
12.2 Середовища	20
12.3 Конфігурація	20
12.3.1 Job resources	20
12.4 Backup policy	20
12.5 Observability	20
13. Нефункціональні вимоги	21
14. Якість і тестування	21
14.1 Піраміда тестів	21
14.2 Критичні test cases	21
14.3 CI pipeline	22
15. План реалізації	22
15.1 MVP release gate	22
15.2 Що можна відкласти без боргу	22
16. Зафіксовані рішення та продуктові defaults	23
16.1 Architecture Decision Records	23
16.2 Зафіксовані MVP defaults	24
16.3 Наступний артефакт	24
Додаток А. Приклад предмета	24
Додаток В. Definition of Done для user story	24
Додаток С. Джерела технічних рішень	25

1. Резюме продукту

Inventory Atlas - самостійно розгортана web-система для обліку фізичних речей і точного відображення місць їх зберігання. Вона поєднує каталог предметів, довільно вкладену структуру локацій і контейнерів, типізовані користувацькі поля, фото, наскрізний пошук, QR та штрих-коди, друк етикеток, історію переміщень і переносимий резервний експорт.

Основний сценарій: за кілька секунд знайти не лише картку камери Pentax LX, а й побачити повний шлях: Склад 1 / бокс Б / коробка А, відкрити фото коробки, відсканувати її код телефоном і переглянути весь вміст.

1	N	2	0
одна система обліку	будь-яка глибина вкладення	типи кодів у MVP	хмарних залежностей для базового запуску

1.1 Проблема

- Звичайна таблиця добре зберігає назви, але погано описує коробку в коробці, історію переміщень і пов'язані фото.
- Фіксований набір колонок швидко застаріває: для фотокамер потрібен байонет і формат плівки, для вінілових платівок - лейбл і каталожний номер, для серверних дисків - інтерфейс і серійний номер.
- Фізичні етикетки без живого каталогу стають неактуальними, а посилання, що містять назву предмета, ламаються після перейменування.
- Публічний каталог корисний для перегляду, але точне місце зберігання та службові поля не повинні автоматично ставати публічними.

1.2 Бачення

Система має відчуватися як зручний особистий каталог, а не як складська ERP. Типові дії виконуються з телефона однією рукою; складні налаштування залишаються в адмінці. Архітектура починається простою Docker Compose інсталяцією, але не закриває шлях до окремого object storage, фонового worker, пошукового рушія чи кількох інстансів API.

Головний принцип

Назва предмета, його статус, фізичне місце, доставка і продаж - різні поняття. Вони моделюються окремо, щоб поле status не перетворилося на суперечливий список на кшталт «в коробці», «продано» і «їде Новою поштою».

1.3 Цілі першого релізу

- Додавати, редагувати, копіювати, архівувати й масово обробляти предмети.
- Створювати довільні типізовані поля без зміни коду або схеми основної таблиці items.
- Будувати дерево складів, кімнат, стелажів, боксів і коробок з необмеженою практичною глибиною.
- Шукати одним рядком по всьому індексованому вмісту та застосовувати типізовані фільтри.
- Працювати з основним і додатковими фото предметів та контейнерів.
- Генерувати QR і Code 128, друкувати шаблони етикеток та сканувати їх камерою телефона.
- Керувати користувачами, ролями, публічністю і журналом змін.
- Експортувати та імпортувати переносимий архів з даними й медіа.

1.4 Нецілі MVP

- Повноцінна бухгалтерія, закупівлі, амортизація та складські документи рівня ERP.

- Нативні iOS або Android застосунки: мобільний web і PWA достатні для першої версії.
- Offline-first синхронізація з конфліктами між пристроями.
- Автоматичне розпізнавання предметів на фото та генеративне заповнення полів.
- Універсальна інтеграція з усіма маркетплейсами без попереднього вибору конкретних каналів.

2. Користувачі, ролі та доступ

2.1 Основні персони

Персона	Потреба	Типовий пристрій
Власник	Повне керування, структура полів, користувачі, backup, інтеграції	Desktop + mobile
Редактор	Швидко додати предмет, фото, перемістити, надрукувати етикетку	Mobile + desktop
Переглядач	Знайти предмет і побачити дозволені деталі	Будь-який браузер
Публічний відвідувач	Перегляд відкритої частини каталогу без входу	Mobile-first

2.2 Рольова модель

Дія	Owner	Admin	Editor	Viewer	Public
Перегляд дозволених даних	Так	Так	Так	Так	Так
Створення та редагування предметів	Так	Так	Так	Hi	Hi
Переміщення і друк етикеток	Так	Так	Так	Hi	Hi
Поля, статуси, шаблони	Так	Так	Hi	Hi	Hi
Користувачі та ролі	Так	Так*	Hi	Hi	Hi
Backup, import, інтеграції	Так	За дозволом	Hi	Hi	Hi

* Admin не може видалити останнього Owner або змінити його роль без підтвердження Owner.

2.3 Політика публічності

Глобальний режим каталогу можна встановити як public або authenticated. Окремо кожен предмет, контейнер і визначення поля має видимість public, authenticated, private або unlisted. Точний storage path за замовчуванням прихований у публічному інтерфейсі, навіть якщо картка предмета публічна.

Безпека фізичних речей	Публікація каталогу не повинна розкривати адресу складу, номер кімнати, коробки або серійний номер. Для цих даних потрібна окрема явна згода на рівні поля чи вузла.
-------------------------------	--

3. Інформаційна модель

3.1 Предмет

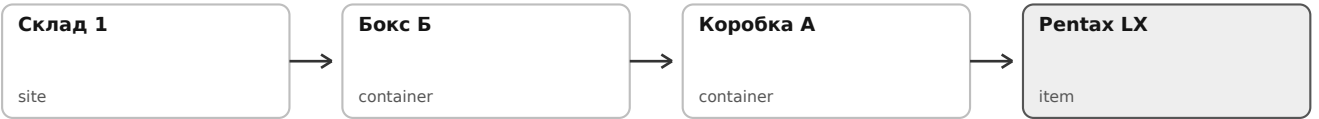
Item має невелике стабільне ядро та розширюється користувацькими полями. Такий поділ залишає швидкими основні запити й одночасно дозволяє описувати різні колекції без міграції БД для кожної нової характеристики.

Група	Поля	Коментар
Стабільне ядро	id, public_id, display_name, description, category_id, lifecycle_status_id, storage_node_id, visibility, created_at, updated_at, version	Контролюється кодом і міграціями
Динамічні атрибути	brand, model, year, condition, serial_number, format, mount та будь-які нові поля	Визначаються в адмінці, мають тип і правила

Група	Поля	Коментар
Зв'язки	tags, media, movements, shipments, listings, audit events	Окремі таблиці й модулі

3.2 Дерево зберігання

Усі фізичні місця зберігання представлені сутністю `StorageNode`. Тип вузла задає семантику: `site`, `room`, `zone`, `rack`, `shelf`, `container` або `custom`. `parent_id` задає ієрархію, а обов'язкові `ltree path` і `depth` матеріалізують маршрут та глибину для швидких `breadcrumbs`, `subtree`-запитів і перевірки циклів. Кожен `ltree label` має формат `n + 32 lowercase hex symbols UUID без дефісів`, наприклад `n018f0a...`; він стабільний між `rename` і переносимий між підтримуваними PostgreSQL versions.



`parent_id` задає ієрархію; `path` і `depth` транзакційно матеріалізують маршрут та глибину.

Move sync оновлює tree/path projections; background job перебудовує лише search vectors.

3.3 Правила дерева

- Не можна зробити вузол нащадком самого себе або власного піддерева: `target.path < @source.path` відхиляється до зміни даних.
- Видалення непорожнього вузла заборонене. Дозволено архівувати або перенести вміст.
- Move бере `SELECT FOR UPDATE` на `source` і `target` та `pg_advisory_xact_lock` на `roots`. Для `cross-root move` обидва `root locks` беруться у `deterministic ID order`. Це серіалізує конкурентні переміщення й мінімізує `deadlock risk`.
- У тій самій Kysely-транзакції оновлюються `parent_id`, `path`, `depth` і `tree_root_id` усього піддерева, `movement`, а також `path_text`, `public_path_text` і `visibility affected item_search rows`. Після `commit` job перебудовує лише `search vectors`; `index_state=stale` виключає рядок з `relevance mode`, але не з каталогу/default search.
- Rename не змінює `storage_nodes.path`, але синхронно оновлює `path_text/public_path_text` усього піддерева та запускає `vector reindex job`, бо людська назва входить у пошуковий документ.
- Item посилається на найближчий `storage node`. Якщо Item перебуває у відправленні, це показується окремим транспортним станом.
- Картка `container` є повноцінною: фото, опис, код, атрибути, дочірні вузли та безпосередні предмети.

3.4 Динамічні поля

Тип поля	Приклад	Фільтрація
text / long_text	Pentax, нотатка про дефект	містить, дорівнює, порожнє
number + unit	1983, 1.2 kg	діапазон, більше, менше
boolean	перевірено	так / ні / не задано
date / datetime	дата купівлі	період
select / multiselect	стан: б/в	одне або кілька значень
url / email	посилання на інструкцію	валідація формату
money	ціна купівлі	сума + валюта
reference	сумісний об'єктив	зв'язок з Item

FieldDefinition містить key, label_i18n, scope, data_type, required, repeatable, default_value, unit, validation rules, option set, searchable, filterable, sortable, public_visibility та display_order. Англійська label є обов'язковою, українська доступна в MVP. Зміна типу поля після появи значень виконується лише через керовану конвертацію з preview.

3.5 Генератор display name

Шаблон назви задається глобально або для категорії. Приклад:

```
{{type}} {{brand}} {{model}} {{condition}}
```

Шаблон використовує обмежену безпечну граматику, а не JavaScript. Порожні токени пропускаються, зайві пробіли стискаються, результат показується в preview. display_name кешується і перебудовується після зміни залежних полів. Унікальне URL-посилання базується на public_id і не змінюється разом з назвою.

3.6 Статуси без змішування контекстів

Життєвий цикл (приклад)	stored reserved sold archived
Фізичне місце	storage node або active shipment
Доставка	draft in transit delivered returned
Продаж	draft listed sold cancelled

Lifecycle status є налаштованим довідником. Рекомендований початковий набір: stored, reserved, lent, for_sale, sold, lost, archived. Доставка і listing мають власні state machine. UI може показувати зведений badge, але БД зберігає джерела стану окремо.

4. Функціональні вимоги

4.1 Каталог і картка предмета

- Grid і table режими, сортування, пагінація, збережені представлення та масові дії.
- Основне фото, галерея, placeholder за категорією, drag-and-drop сортування фото.
- Повний storage breadcrumb, швидкий перехід до контейнера, історія переміщень.
- Унікальний стабільний URL, копіювання посилання, керована публічність.
- Duplicate as new для серій однотипних речей; явне очищення серійного номера при копіюванні.
- Optimistic concurrency: якщо картку паралельно змінив інший користувач, UI не перезаписує дані мовчки.

- Bulk action передає id + expected version для кожного Item. Кожен Item комітиться окремо; відповідь містить succeeded, conflicted і failed з актуальною version/diff. Конфлікт одного Item не відкочує успішні зміни інших, а повтор з тим самим idempotency key не дублює ефект.

4.2 Додавання та редагування

Форма генерується зі schema definitions. На мобільному вона має quick mode: фото, категорія, коротка назва, контейнер і Save. Решту полів можна дозаповнити пізніше. На desktop доступний повний режим із секціями, підказками, keyboard navigation і sticky action bar.

4.3 Пошук і фільтри

Механізм	Поведінка
Наскрізний пошук	Назва, опис, категорія, текстові динамічні поля, теги, коди та дозволений storage path
Type-aware filters	Різні оператори для text, number, date, boolean, select та reference
Комбінації	AND між групами, OR усередині multiselect; активні фільтри видно як chips
Saved views	Назва, власник, спільність, колонки, сортування, фільтр
Помилки вводу	Trigram similarity знаходить близькі написання; exact code має найвищий пріоритет
Pagination	Default sort: item_updated_at DESC, item_id DESC зі stable cursor; relevance mode - лише перші 500 результатів

4.4 Фото і медіа

- Приймати JPEG, PNG, WebP і HEIC/HEIF. HEIC/HEIF декодує серверний image build із libvips + libheif; capability перевіряється startup healthcheck.
- Зберігати оригінал і похідні thumbnail/small/medium; видаляти EXIF GPS за замовчуванням.
- Перевіряти MIME за вмістом, ліміти розміру та pixel count; обробку виконувати поза HTTP request.
- Media storage доступне через адаптер: local volume у базовій інсталяції, S3-compatible у масштабованій.
- Експорт містить оригінали, manifest із checksum та зв'язки з предметами й контейнерами.

4.5 Етикетки, QR і штрих-коди

Коди створюються для Item і StorageNode. QR містить короткий HTTPS URL зі стабільним opaque token. Code 128 містить компактний внутрішній code, який scanner route перетворює на сутність. Назва не кодується як ідентифікатор.

Можливість	Вимога MVP
Формати	QR та Code 128
Шаблони	Розмір сторінки, label grid, поля, шрифти, logo optional, preview
Вихід	Векторний PDF для друку; одиночна PNG/SVG картинка коду за потреби
Batch print	Вибір предметів/контейнерів, кількість копій, starting position на аркуші
Scanner	Камера, ліхтарик якщо API доступний, ручне введення, history останніх сканів
Після скану	Відкрити картку, показати вміст, перемістити сюди, почати інвентаризацію

Нативний BarcodeDetector у браузерях ще має неповну підтримку, тому він використовується лише як прискорення. Основний кросбраузерний шлях - ZXing-based decoder. Доступ до камери потребує HTTPS або localhost.

4.6 Режим інвентаризації

Для обраного контейнера система створює count session. Користувач сканує очікувані предмети, а UI розділяє їх на found, missing, unexpected і moved. Завершення сесії створює immutable summary та пропонує, але не виконує автоматично, виправлення місць зберігання.

4.7 Імпорт, експорт і backup

Потрібні два різні механізми:

- Portable export - versioned ZIP для перенесення між інсталяціями: manifest.json, JSONL/JSON data, media/, checksums, schema version і короткий README.
- Operational backup - регулярний pg_dump плюс копія media volume/object storage. Це аварійне відновлення конкретної інсталяції, а не формат обміну.
- Import працює через validate -> dry run -> conflict report -> apply. Підтримує replace, skip і create сору для конфліктів public_id.
- Після імпорту система звіряє кількість сутностей, checksums медіа та будує пошуковий індекс заново.

4.8 Доставка

Shipment може містити окремі items та/або containers. Якщо відправлено контейнер, його нащадки отримують effective in-transit indicator без дублювання рядків. Перший етап - ручна служба, tracking number, дати, recipient note і журнал статусів. Другий - provider adapters для Нової пошти та Укрпошти.

4.9 Продажі

Listing моделюється окремо від Item. Базовий етап: статус, ціна, валюта, майданчик, URL, чернетка опису, дата публікації. Автоматична публікація додається тільки для вибраних майданчиків після перевірки їх API, авторизації, лімітів і правил категорій.

5. UX та інформаційна архітектура

5.1 Основна навігація

Розділ	Зміст
Catalog	Усі предмети, saved views, categories, bulk actions
Storage	Дерево локацій і контейнерів, вміст, переміщення
Scan	Повноекранна камера, ручне введення, історія
Add	Quick add item/container, photo-first workflow
Shipments	Відправлення, tracking timeline, exceptions
Listings	Чернетки й активні оголошення
Admin	Fields, categories, statuses, users, labels, integrations, import/export

5.2 Mobile-first, але не mobile-only

- Mobile: bottom navigation, floating Add/Scan action, одна колонка, великі touch targets, camera-first flows.
- Desktop: sidebar, data table, multi-column filters, keyboard shortcuts, bulk edit і drag-and-drop дерева.
- Ti самі маршрути й domain components; різні layout shells, а не дві окремі програми.
- Scanner і швидке додавання працюють як installable PWA; offline shell допускається, але offline mutations не входять у MVP.

5.3 Ключові сценарії

Сценарій	Кроки	Успіх
Знайти річ	Search -> result -> item -> breadcrumb	Місце і фото видно до 3 дій
Додати камеру	Add -> photo -> category -> fields -> container -> save	Чернетку можна створити менш ніж за хвилину
Підписати коробку	Storage node -> label -> template -> print	PDF має точний розмір і сканується
Перемістити коробку	Scan source -> Move -> scan destination -> confirm	Історія й шляхи нащадків оновлені
Перенести інсталяцію	Export -> new instance -> dry-run import -> apply	Кількість і checksums збігаються

5.4 Дизайн-система

Рекомендація - PrimeVue у styled mode з власним набором semantic design tokens. Він дає готові складні компоненти для таблиць, фільтрів, форм і dialog, а token architecture дозволяє міняти палітру, радіуси й щільність без переписування domain components.

Уся бізнес-розмітка використовує внутрішні компоненти AppButton, AppField, AppTable і AppDialog. Сторінки не залежать напряду від PrimeVue markup, тому в майбутньому можна змінити дизайн-систему або додати теми без переробки domain logic. Єдина тема MVP - сучасна, mobile-first, доступна й придатна для touch UX.

ADR-007

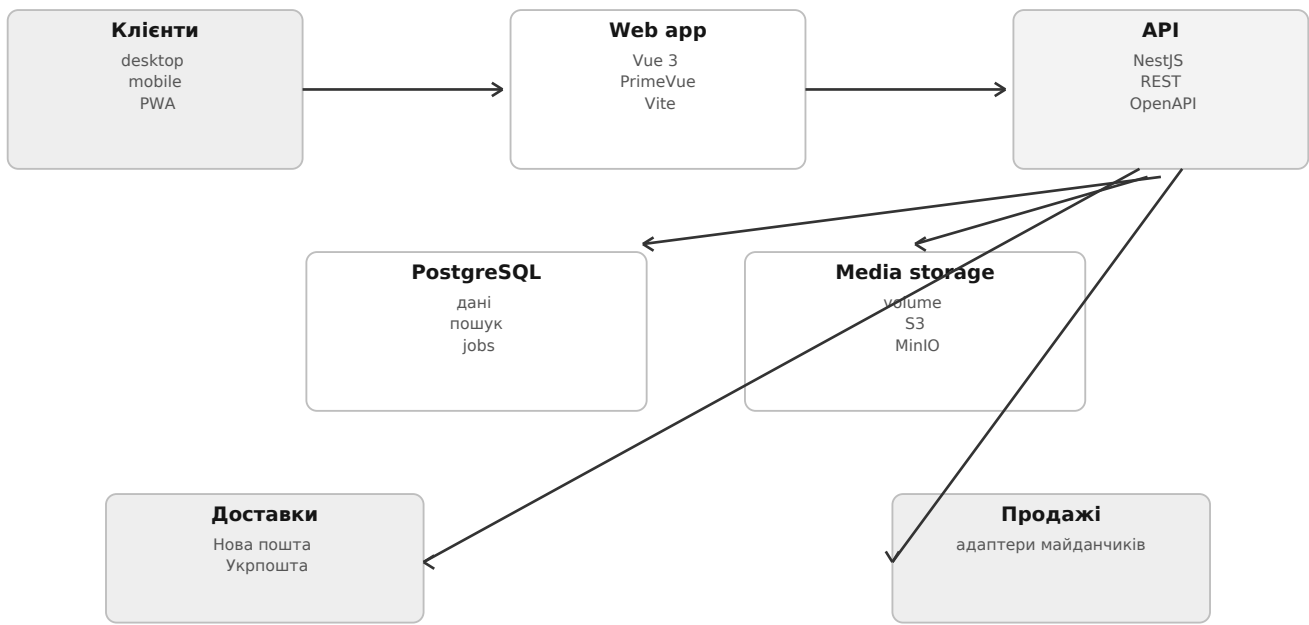
PrimeVue + власні design tokens. Bootstrap не є забороненим, але не обирається базою: для data-heavy адмінки довелося б окремо добирати таблицю, advanced filters і частину form components.

5.5 Локалізація

English є source locale і мовою за замовчуванням для UI, API documentation, seed data та системних довідників. Повна українська локалізація входить у release gate MVP і доступна в language selector. Вибір зберігається у профілі користувача, а для anonymous UI - у cookie/local preference; автоматичне визначення browser locale може лише запропонувати зміну, але не змінює default locale.

Усі UI strings використовують стабільні i18n keys; заборонені hardcoded тексти в domain components. FieldDefinition, category, lifecycle status і option values підтримують локалізовані labels: English required, Ukrainian optional із fallback на English. Власні значення користувача не перекладаються автоматично.

6. Технічна архітектура



6.1 Рекомендований стек

Шар	Вибір	Причина
Frontend	Vue 3, SFC, Composition API, modern JavaScript, Vite, checkJs	JS syntax; статична перевірка через JSDoc і generated contract types
UI	PrimeVue + internal UI facade + design tokens	Data-heavy компоненти, теми, можливість unstyled mode
State	Pinia для session/UI; Vue Query для server cache	Не змішувати локальний стан із даними API
Backend	Node.js 24 LTS, NestJS, TypeScript, Fastify adapter	OOP, DI, модулі, SOLID; LTS runtime; масштабування
API	REST /api/v1 + OpenAPI	KISS, проста діагностика, клієнти генеруються зі spec
Database	PostgreSQL	Транзакції, дерево, typed values, JSONB, FTS, GIN/trigram
Data access	Prisma CRUD + Kysely specialized SQL; trx-aware write ports	Одна business transaction використовує один client; ports приєднуються до source trx
Media	local volume через StorageAdapter; S3/MinIO optional	Простий старт і безболісне масштабування
Jobs	PostgreSQL queue: FOR UPDATE SKIP LOCKED	Без Redis; retry, lease, idempotency та керована concurrency
Labels	qrcode, bwip-js, PDFKit	QR, Code 128 та повторюваний server-side PDF
Images	Sharp + custom libvips/libheif image	Орієнтація, HEIC, thumbnails, WebP, metadata policy
Deployment	Docker Compose + Caddy reverse proxy	Одна команда, HTTPS, homelab-friendly

HEIC distribution note: self-hosted local build лишається цільовим сценарієм. Перед публікацією prebuilt image з HEVC-enabled libheif у public registry власник релізу має окремо перевірити codec/patent licensing та distribution policy; це engineering release gate, а не юридичний висновок.

6.1.1 Власність таблиць і транзакційна межа

Прийнято уточнений варіант A1-1/E1: одна бізнесова транзакція використовує рівно один data-access client. StorageMoveService відкриває Kysely transaction і передає той самий trx handle у Storage, SearchProjection та Outbox. Catalog/Schema mutation відкриває Prisma transaction і передає її в ті самі вузькі write ports. SearchProjectionPort.writeSync(trx, ...) та OutboxPort.enqueue(trx, ...) мають дві реалізації над однаковим parameterized SQL contract: Kysely adapter приймає Kysely trx, Prisma adapter - Prisma.TransactionClient і виконує statement через \$executeRaw. Жоден port не відкриває власної транзакції.

Таблиці	Модуль-власник	Клієнт доступу
items, media relations, tags, shipments, listings	Catalog / Media / Integrations	Prisma
field_definitions, field_options, categories, lifecycle_statuses, attribute_values	Schema / Catalog	Prisma
users, sessions, invitations, audit_events	Auth / Audit	Prisma
storage_nodes, movements	Storage	Kysely only
item_search	Search володіє reads; writeSync(trx) port доступний source modules	Kysely reads; port використовує current Kysely або Prisma trx
jobs	Jobs	Kysely only
outbox	Infrastructure володіє reads/claim; enqueue(trx) port доступний source modules	Kysely reads; port використовує current Kysely або Prisma trx

Prisma і Kysely мають окремі явно обмежені pools; connection budget задається на process і перевіряється startup validation. Compact profile стартує з лімітом 12 application connections, expanded api + worker - 24 без окремого capacity review. Фактичні pool sizes і запас до PostgreSQL max_connections обов'язково перевіряються на першому load test та коригуються до production deployment.

6.2 Чому TypeScript на backend

Вимога про звичайний JavaScript стосується frontend. Для backend рекомендується TypeScript: NestJS спирається на decorators, dependency injection і DTO contracts, а типи зменшують ризик у динамічній моделі полів, імпорті та інтеграціях. Це не змінює формат REST API і не переводить Vue components на TypeScript.

6.2.1 Контрактна безпека JavaScript frontend

ADR-006 лишає .js/.vue syntax, але не лишає API boundary нетипізованою. Єдине джерело контракту - backend OpenAPI document, згенерований з NestJS DTO. Один deterministic codegen step створює frontend declaration/JSDoc types і Zod runtime schemas з тієї самої OpenAPI revision. Hand-written duplicate Zod payload schemas заборонені. Vite/CI запускає checkJs, а field-engine використовує generated discriminated schemas за data_type. OpenAPI diff або codegen drift зупиняє CI.

6.3 Модульний моноліт

Перший реліз - не microservices, а модульний моноліт. Кожен модуль має controller, application service, domain logic, repository interface і infrastructure adapter лише там, де це виправдано. Межі модулів залишають можливість винести Search, Media Jobs або Integrations окремо, якщо навантаження цього вимагатиме.

Модуль	Відповідальність
Auth / Users	sessions, roles, invitations, password reset
Catalog	items, categories, tags, lifecycle
Schema	field definitions, options, validation, display templates
Storage	tree, paths, moves, inventory sessions
Media	uploads, variants, metadata, storage adapter
Search	index document, filters, ranking, saved views

Модуль	Відповідальність
Labels / Scan	codes, templates, batches, token resolution
Portability	export, import, backup metadata, validation
Shipments	providers, tracking, timelines
Listings	channels, drafts, state mapping
Audit	append-only security and domain events

6.4 SOLID, OOP, KISS і DRY на практиці

- Single responsibility: label rendering не знає про HTTP; shipment provider не оновлює Item напряду.
- Dependency inversion: MediaService залежить від StoragePort, а не від local filesystem чи S3 SDK.
- Open/closed: нова служба доставки додається реалізацією ShipmentProvider без switch по всьому коду.
- KISS: REST, modular monolith, PostgreSQL і один compose stack; Redis, OpenSearch та microservices не додаються наперед.
- DRY без over-engineering: спільні правила в domain services і shared schemas, але не універсальний BaseRepository для несхожих агрегатів.
- OOP використовується для поведінки й інваріантів; прості DTO, mappers і pure functions не перетворюються на класи без потреби.

7. Модель даних

7.1 Основні сутності

Сутність	Ключові поля	Зв'язки
items	id, public_id, category_id, display_name, status_id, storage_node_id, visibility, version	attributes, media, tags, movements
storage_nodes	id, parent_id, path, depth, tree_root_id, node_type, title, code, visibility, version	children, items, attributes, media
categories	id, parent_id, key, label_i18n, display_order, archived_at	items, child categories
lifecycle_statuses	id, key, label_i18n, color_token, display_order, archived_at	items
field_definitions	scope, key, label_i18n, type, validation_json, searchable, filterable, visibility	options, typed values
field_options	id, field_id, key, label_i18n, display_order, archived_at	attribute values
attribute_values	subject_type, subject_id, field_id, typed value columns	definition; unique rules
item_search	item_id, search vectors, path text, public/auth attrs, visibility, filters, index_state, item_updated_at, indexed_at	one materialized row per item
media_assets	storage_key, mime, bytes, width, height, checksum	item_media, node_media
entity_codes	entity, format, public_token, short_code, revoked_at	scan resolution
movements	subject, from_node, to_node, shipment_id, actor, occurred_at	append-only history
shipments	provider, tracking_no, state, timestamps, raw_state	items, nodes, events
listings	channel, external_id, state, price, currency, url	item or bundle
audit_events	actor, action, entity, before, after, request_id, timestamp	security/admin trace
jobs	type, payload, state, attempts, available_at, locked_at, progress	media, subtree reindex, import/export

7.2 Представлення динамічних значень

Рекомендований компроміс - EAV з типізованими колонками: `value_text`, `value_number`, `value_boolean`, `value_date`, `value_datetime`, `value_json` і `option_id`. Для кожного рядка заповнюється рівно одне релевантне поле. Це складніше за один JSON blob, але дає коректні діапазони, сортування, validation та індекси.

ADR-004

JSONB використовується для validation config, snapshots і зовнішніх payload, але не як єдине сховище всіх атрибутів. Поля, по яких користувач фільтрує і сортує, мають типізовані значення.

7.3 Ідентифікатори

- Internal id - UUIDv7 або UUID, що не залежить від назви й середовища.
- public_id - окремий stable opaque identifier у URL.
- short_code - людиночитний унікальний код для Code 128 та ручного вводу, наприклад BX-00A17.
- display_name - похідне поле; slug може бути декоративним, але routing завжди спирається на public_id.

7.4 Цілісність

- Унікальність field_definition.key у межах scope або category.
- Check constraint гарантує рівно одне typed value у attribute_values.
- Storage tree захищене path-based cycle check, row locks і root advisory lock; move атомарно оновлює path, depth і tree_root_id піддерева.
- Item разом з усіма attribute_values є одним aggregate. Будь-яка зміна core або dynamic field інкрементує items.version; паралельні зміни різних полів одного item вважаються конфліктом і повертають diff.
- Code token не перевикористовується після revoke.
- Audit і movement history append-only для звичайних користувачів.
- Soft archive застосовується до предметів і вузлів; media hard delete виконується delayed cleanup job після retention period.

7.5 item_search як read model

Typed EAV лишається source of truth. item_search є відновлюваним materialized read model: один рядок на Item з public та authenticated search projections, дозволеним path preview, effective visibility, гарячими scalar filters та attrs JSONB лише для filterable dynamic fields. Private values не входять у жодну проєкцію; Owner/Admin exact search читає attribute_values з role check. Значення нормалізуються за data_type; multiselect зберігається як array, number/date - у канонічному типі.

Колонка	Призначення
item_id	PK/FK; один search row на предмет
search_vector / public_search_vector	authenticated (public+authenticated) і public-only проєкції; private values виключені
path_text / public_path_text	повний breadcrumb; public projection порожня за замовчуванням
visibility	денормалізована effective visibility для безпечного відбору
category_id / lifecycle_status_id	гарячі scalar filters без EAV joins
attrs / public_attrs jsonb	authenticated і public-only проєкції filterable values; private values виключені
index_state	ready або stale; stale впливає лише на relevance mode
item_updated_at	business timestamp copied from Item; stable sort cursor
indexed_at	technical rebuild timestamp; не змінює default ordering

7.5.1 Visibility x search policy

Рівень	Public search	Viewer / Editor	Owner / Admin
public	public vector/attrs	так	так
authenticated	ні	search_vector / attrs	так
private	ні	ні; не індексується	exact EAV match з role check
unlisted item/node	немає в search/list; direct token/URL	немає в search/list; direct URL	видимий у search/list

Unlisted є discoverability policy рівня Item/StorageNode, а не field value. Token/URL resolution все одно застосовує visibility та authorization; unlisted не означає обхід private permissions.

8. API design

8.1 Принципи

- Versioned base path /api/v1; JSON; UTF-8; OpenAPI є контрактом.
- Cursor pagination для великих списків; limit має верхню межу. Default cursor кодує (item_updated_at, item_id) у descending order і є opaque для клієнта; indexed_at у курсор не входить.
- Relevance mode сортує за rank, item_updated_at, item_id, але обмежується першими 500 результатами / 20 сторінками; deep pagination для relevance не є вимогою MVP.
- Problem Details для помилок із machine-readable code, field errors і request_id.
- Idempotency-Key для import jobs, label batches і створення shipment.
- If-Match або version у mutation для optimistic concurrency.
- Файли завантажуються multipart у базовому режимі або presigned URL у S3 mode.

8.2 Основні endpoints

Метод і шлях	Призначення
GET /items	Search, filters, sort, cursor, fields projection
POST /items	Створення item з атрибутами і destination node
GET/PATCH /items/{id}	Картка та versioned update
POST /items/bulk-actions	Per-item expected version; succeeded/conflicted/failed report
POST /items/{id}/move	Атомарне переміщення з movement event
GET/POST /storage-nodes	Дерево, children, створення вузла
POST /storage-nodes/{id}/move	Зміна parent із cycle check
GET/POST /field-definitions	Schema-driven admin forms
POST /media/uploads	Upload session або multipart
POST /label-batches	PDF job і параметри шаблону
GET /scan/{token}	Resolve code з урахуванням visibility
POST /inventory-sessions	Count session для контейнера
POST /exports /imports	Async portable archive jobs
GET/POST /shipments	Ручний tracking та provider integration
GET/POST /listings	Чернетки і channel mapping
GET/POST /admin/users	Користувачі та ролі

8.3 Події й інтеграції

item_search є похідним read model, тому кожне джерело денормалізованих даних має явний інвалідатор. Sync updates виконуються в транзакції source change; vector jobs створюються через outbox, є resumable та показують progress.

Подія	Що змінюється	Обсяг	Режим
ItemCreated / AttributeChanged	увесь item_search row	1 item	sync, та сама transaction
NodeMoved	path_text, public_path_text, visibility; vectors	subtree	path/visibility sync; vectors job
NodeRenamed	path_text, public_path_text; обидва vectors	subtree	path sync; vectors job
NodeVisibilityChanged	visibility і всі public projections	subtree	sync
CategoryRenamed	обидва vectors	items категорії	job
FieldDefinitionChanged	attrs/public_attrs і vectors	items з field	job + UI warning
FieldOptionLabelChanged	обидва vectors	items з option	job
ItemVisibilityChanged	visibility і public projections	1 item	sync

Sync-запис projection і постановка job виконуються через SearchProjectionPort.writeSync(trx, ...) та OutboxPort.enqueue(trx, ...) у транзакції source change незалежно від того, Prisma чи Kysely володіє source-таблицею. Port використовує переданий transaction handle, не створює окремого commit і тому зберігає атомарність усіх восьми invalidator paths.

Для subtree change path projections та visibility завжди коректні після commit. index_state=stale означає лише, що text rank ще не перебудований: row лишається у catalog/default search, але тимчасово не бере участі в relevance mode. Import буде index batch-ами й не оголошує результат повним до завершення job.

Для зовнішніх webhook та надійного запуску jobs застосовується transactional outbox. Worker забирає записи через FOR UPDATE SKIP LOCKED, має lease/heartbeat, retry з backoff, dead-letter state та idempotency key. Це не дозволяє зміні БД і постановці події розійтися.

9. Пошук, фільтрація та масштабування

9.1 Пошуковий документ

Для кожного Item фізично існує один рядок item_search з public та authenticated проєкціями: display_name, description, category, tags, дозволені searchable attributes, short codes, storage path, effective visibility, hot scalar filters та attrs JSONB. Private field values не індексуються; Owner/Admin exact lookup звертається до typed EAV з role check. Unlisted items/nodes виключені з search і listings для Public/Viewer/Editor, але resolvable за дозволеням direct URL/token; Owner/Admin бачить їх у результатах.

Через змішані українські, російські та англійські назви перший реліз використовує simple normalization, lower-case, unaccent де доречно та trigram. У search vector входять одночасно English і Ukrainian labels category, lifecycle status та field options, наприклад used і б/в; user-entered value додається один раз. Мовний stemming не повинен погіршувати серійні номери й моделі на кшталт HUC10121CLAR1200.

Default search order - item_updated_at DESC, item_id DESC зі stable cursor. indexed_at не впливає на business ordering. Relevance є окремим bounded mode: rank DESC, item_updated_at DESC, item_id DESC, максимум 500 results. Це явний UX-компроміс MVP, а не прихована заміна cursor pagination на OFFSET.

9.2 Індекси

Запит	Індекс
За public_id / short_code	Unique B-tree
Full-text	GIN(search_vector) і GIN(public_search_vector)
Dynamic filters	GIN(attrs/public_attrs jsonb_path_ops) + expression indexes для 5-10 гарячих keys
Схожість і contains	GIN(path_text gin_trgm_ops) та pg_trgm expression indexes
Source-of-truth ranges	B-tree(field_id, value_number/date) на attribute_values
Дерево	GiST(path ltree), B-tree(parent_id), B-tree(tree_root_id, depth)
Активні shipment/listing	Partial indexes за state

9.2.1 SQL policy

Prisma читає лише Prisma-owned tables; читання Kysely-owned item_search, outbox/jobs і storage tables через Prisma заборонене. Storage, Search, Jobs і Outbox repositories використовують Kysely та parameterized SQL для ltree operators, CTE/batch updates, JSONB path filters і SKIP LOCKED. Виняток лише для атомарного запису: Prisma-owned Catalog/Schema transaction може змінити item_search або outbox через SearchProjectionPort/OutboxPort, які приймають Prisma.TransactionClient і виконують узгоджений parameterized statement через \$executeRaw у тій самій transaction. Kysely-owned source використовує симетричний adapter з тим самим port contract. PostgreSQL-specific migrations зберігаються як reviewable SQL, integration tests виконуються лише на real PostgreSQL.

9.3 Пороги еволюції

Рівень	Орієнтир	Архітектура
Personal	до 100 тис. items, один homelab	PostgreSQL + local media; один API
Expanded	великі imports, багато фото, кілька users	S3/MinIO, окремий worker, read replica за потреби
Search-heavy	мільйони records або складний ranking	Outbox -> Meilisearch/OpenSearch; PostgreSQL лишається source of truth
Multi-instance	HA або зовнішній сервіс	Shared sessions/DB, object storage, load balancer

Не масштабувати завчасно Поява adapter interfaces не означає, що MinIO, Redis або OpenSearch мають бути обов'язковими контейнерами в першому docker-compose. Базовий запуск залишається легким.

10. Авторизація, безпека та аудит

10.1 Authentication

Для browser UI рекомендовані opaque server-side sessions у Secure, HttpOnly, SameSite cookie. Session token зберігається в БД лише у hashed вигляді, має inactivity timeout, absolute expiry і можливість revoke. Для automation/API clients пізніше додаються personal access tokens з scopes.

10.2 Обов'язкові контрзаходи

- Argon2id для паролів; invitation і reset tokens одноразові та короткоживучі.
- CSRF protection для cookie-auth mutations, strict CORS і same-origin deployment за замовчуванням.
- Rate limit на login, scan token resolution, import і public search.
- Validation усіх DTO; параметризовані SQL; жодних довільних template expressions.
- MIME sniffing, image decode limits, filename normalization, quarantine failed media.
- Secrets через environment/Docker secrets; контейнери non-root; database не публікується назовні.
- Регулярне dependency scanning та pinning образів за digest у production manifest.

10.3 Audit

Audit event фіксує actor, action, entity, timestamp, request_id і безпечний diff. Паролі, session tokens, API keys та повні provider payload з персональними даними не потрапляють у журнал. Owner може експортувати audit, але не редагувати записи через UI.

10.4 Threat model highlights

Ризик	Захист
QR сфотографований сторонньою особою	Visibility check, revoke token, optional unlisted share expiry
Публічний каталог видає адресу	Field/node visibility; exact path private by default
Шкідливий import ZIP	Size limits, path traversal defense, schema validation, dry run, isolated staging
Підміна delivery webhook	Signature/IP verification де provider підтримує; poll reconciliation
Втрата media volume	Checksummed export + operational backup + restore drill
Паралельне редагування	Optimistic version and conflict UI

11. Інтеграції

11.1 Служби доставки

ShipmentProvider interface нормалізує createShipment optional, getTracking, cancel optional, printCarrierLabel optional і mapStatus. Сирий provider status зберігається поруч із нормалізованим статусом для діагностики. API keys шифруються at rest або передаються через secret store.

Provider	Можливість	Рекомендований етап
Нова пошта	API key, створення ЕН, довідники, tracking, маркування	P1: tracking; P2: create/print
Укрпошта	Bearer auth, shipment API, tracking, label	P1: tracking; P2: create/print
Generic	Ручний carrier + tracking URL template	MVP

Polling запускається фоновими jobs із exponential backoff. Завершені shipment опитуються рідше або не опитуються. Provider outage не блокує catalog API.

11.2 Майданчики продажу

SalesChannel interface повинен підтримувати capabilities: draft, publish, update, unpublish, syncStatus. Оскільки набір майданчиків ще не визначено, MVP зберігає listing metadata і готує опис/фото, але не обіцяє автоматичну публікацію. Кожен connector додається після окремого feasibility review.

11.3 Правила інтеграцій

- Жодна зовнішня система не є source of truth для inventory item.
- Mapінг зовнішніх статусів версонується і має unknown fallback.
- Credentials ніколи не входять до portable export за замовчуванням.
- Rate limits, retries і idempotency реалізуються у provider adapter, а не в UI.
- Видалення connector не видаляє shipment/listing history.

12. Розгортання й експлуатація

12.1 Базовий Docker Compose

Сервіс	Призначення	Обов'язковий
web	Caddy: HTTPS, static Vue build, /api proxy	Так
api	NestJS; compact queue runner; media decode лише в capped child process	Так
db	PostgreSQL із persistent volume	Так
worker	Той самий custom image з libvips/libheif; isolated media process	Profile expanded
minio	S3-compatible media storage	Profile expanded

12.2 Середовища

Environment	Особливості
Development	Hot reload, seeded demo data, local media, exposed DB only on localhost
Test/CI	Ephemeral PostgreSQL, migrations from zero, integration and E2E
Production homelab	Pinned images, TLS, volumes, healthchecks, scheduled backups
Dedicated server	Same compose or OCI deployment, external object storage optional

12.3 Конфігурація

Усі environment variables документуються в .env.example без секретів. Startup validation зупиняє застосунок при відсутньому SESSION_SECRET, некоректному public URL або media backend. Image capability healthcheck перевіряє JPEG/PNG/WebP/HEIC decode на fixtures. Database migration запускається окремим one-shot command до старту нової версії API.

12.3.1 Job resources

Compact profile claim-ить jobs у API з global concurrency 1, але Sharp/libheif ніколи не виконується в API process. Media job запускає одноразовий child process через Linux resource-limit wrapper з окремим address-space/RSS cap, timeout, pixel/input limits і sharp concurrency=1; crash/OOM child переводить job у retry, не зупиняє API. Expanded profile виконує той самий processor в окремому worker container: media concurrency 1 default, максимум 2 після load test. Import/export та reindex мають окремі per-type limits. Queue використовує FOR UPDATE SKIP LOCKED, lease recovery і graceful shutdown.

12.4 Backup policy

- Щоденний pg_dump і snapshot media; retention приклад: 7 daily, 4 weekly, 6 monthly.
- Копія backup поза сервером за правилом 3-2-1, якщо дані справді цінні.
- Checksums і machine-readable backup manifest.
- Щоквартальний restore drill у чистому environment; успіх фіксується в admin health page.
- Portable export не замінює автоматичний operational backup.

12.5 Observability

Structured JSON logs з request_id, health endpoints для API/DB/storage, metrics для request latency, job failures, storage usage, import/export duration і provider errors. У базовій інсталяції достатньо logs + health; Prometheus/OpenTelemetry додаються профілем.

13. Нефункціональні вимоги

Категорія	Ціль
Performance	p95 item read < 300 ms; default search < 800 ms на 100 тис. items у reference homelab після warm-up
Mobile	Основні сценарії від 360 px; camera flow на iOS Safari і Android Chrome
Accessibility	WCAG 2.2 AA target; keyboard, focus, labels, contrast, reduced motion
Availability	Graceful failure зовнішніх інтеграцій; catalog працює без carrier API
Portability	Чистий import portable archive у підтримувану новішу версію
Localization	English default/source locale; повна Ukrainian locale в MVP; Europe/Kyiv display timezone
Browser support	Останні 2 major Chrome/Edge/Firefox/Safari; scanner має fallback
Data integrity	Transactional moves, version conflicts, checksummed media exports
Maintainability	Module boundaries, OpenAPI, migrations, ADR, automated tests

Performance numbers є acceptance targets, а не гарантією для будь-якого диска чи NAS. Перед релізом потрібен зафіксований test dataset і reference hardware profile.

14. Якість і тестування

14.1 Піраміда тестів

Рівень	Що перевіряємо
Unit	Display template, validation, status transitions, path/cycle rules, provider mapping
Integration	Repositories на real PostgreSQL, transactions, indexes, migrations, queue
API contract	OpenAPI schema, auth scopes, errors, pagination, concurrency
E2E	Add, search, move, label, scan resolve, export/import, role restrictions
Visual	Mobile/desktop breakpoints, themes, print preview, placeholder/media
Physical	Надруковані QR/Code 128 різних розмірів, iPhone/Android, погане освітлення
Recovery	Backup restore та portable round-trip на чистій інсталяції

14.2 Критичні test cases

- Спроба перемістити Box В всередину власної дочірньої Box А відхиляється без часткового update.
- Паралельні move А під В і В під А серіалізуються; після завершення дерево лишається acyclic і connected.
- Після одиничного save Item одразу знаходиться; під час subtree/import reindex UI показує indexing state і job відновлюється після restart.
- Rename контейнера синхронно оновлює breadcrumb усіх вкладених Items; вони не зникають із default/public catalog під час vector reindex.
- Зміна public_visibility/searchable/filterable поля прибирає його значення з public projections після завершення контрольованого reindex.
- Viewer не може підтвердити private field value через text search, typed filter або result count; Owner/Admin exact lookup проходить role check.
- Unlisted Item відсутній у Public/Viewer/Editor search і listings, але дозволений direct URL/token працює; Owner/Admin бачить його в search.
- Зміна brand перебудовує display_name, але item URL і QR продовжують працювати.
- Public user бачить картку, але не бачить private field і storage path.
- Export -> import відновлює кількість items/nodes/media, ordering фото й checksums.
- Невідомий carrier status зберігається як raw, а normalized стає unknown, не delivered.

- Два редактори змінюють одну картку: другий отримує conflict і diff.
- Bulk action комітить non-conflicting Items і повертає per-item conflict report без повторного застосування succeeded entries.
- Storage move, movement, synchronous path projections і outbox job використовують один Kysely transaction handle; Prisma query у цій transaction заборонена тестом ownership boundary.
- Catalog/Schema transaction змінює source rows, item_search projection і outbox через Prisma-aware write ports в одному Prisma transaction; ports не відкривають окремої transaction, а rollback source change відкочує projection та outbox разом.
- Видалення field definition з даними вимагає archive або explicit destructive migration.
- English UI є default у чистій інсталяції; Ukrainian можна вибрати без restart, усі MVP screens і validation messages перекладені.
- API payload із неправильним runtime type відхиляється frontend boundary до mutation і backend DTO validation на сервері.

14.3 CI pipeline

На кожен pull request: formatting/lint, unit, integration, OpenAPI diff, frontend build, migration from clean DB, E2E smoke, dependency audit і container build. На release: versioned images, SBOM, migration artifact, changelog і backup compatibility note.

15. План реалізації

Етап	Результат	Пріоритет	Оцінка
0. Discovery	Defaults, fixtures, label proof, reference hardware	P0	1 тиж.
1. Foundation	Monorepo, Vue/Nest, PostgreSQL, auth, i18n, Docker, CI	P0	3-4 тиж.
2. Core catalog	Items, fields, schema forms, display template, media/HEIC	P0	5-7 тиж.
3. Storage	Itree path/depth, safe moves, history, containers	P0	3-4 тиж.
4. Find	item_search, visibility projections, filters, cursors, saved views, bulk	P0	4-6 тиж.
5. Labels and scan	QR, Code 128, templates, PDF, PWA scanner	P0	3-4 тиж.
6. Portability	Export/import, backup docs, restore and round-trip	P0	3-4 тиж.
7. Inventory sessions	Count workflow, missing/unexpected reports	P1	2-3 тиж.
8. Shipments	Manual shipment, then tracking adapters	P1	2-4 тиж.
9. Listings	Manual metadata, then selected connector	P2	2-5 тиж.
10. Scale	Worker profile, S3/MinIO, external search by metrics	P2	2-4 тиж.

Оцінки наведені для одного досвідченого full-time розробника, включають automated tests, повну English/Ukrainian localization і документацію, але не довгі зовнішні API approvals. MVP (етапи 0-6) - орієнтовно 22-30 тижнів. Це planning range, а не optimistic best case. Для part-time розробки календарний строк масштабується пропорційно доступності; кожен етап завершується deployable increment.

15.1 MVP release gate

MVP вважається готовим, коли один користувач може розгорнути систему з README, вибрати English або Ukrainian UI, створити структуру Warehouse -> Box -> Case, додати предмет із custom fields і фото, знайти його, надрукувати й відсканувати етикетку, безпечно перенести коробку, обмежити публічний доступ і виконати перевірений export/import.

15.2 Що можна відкласти без боргу

- Real-time carrier tracking, якщо shipment model і provider port уже визначені.

- Marketplace publish, якщо Listing відокремлений від Item.
- S3/MinIO, якщо MediaService одразу працює через StorageAdapter.
- External search, якщо search indexing запускається через події, а PostgreSQL залишається source of truth.
- Додаткові UI themes, якщо domain components не залежать напряду від PrimeVue markup у pages.

16. Зафіксовані рішення та продуктові defaults

16.1 Architecture Decision Records

ADR	Рішення	Статус
ADR-001	Modular monolith замість microservices	Прийнято
ADR-002	PostgreSQL як source of truth	Прийнято
ADR-003	StorageNode: parent_id + ltree path/depth; root advisory lock для move	Прийнято
ADR-004	Typed EAV для динамічних фільтрованих полів	Прийнято
ADR-005	REST + OpenAPI	Прийнято
ADR-006	Vue JavaScript + checkJs/JSDoc/OpenAPI/Zod; NestJS TypeScript	Прийнято
ADR-007	PrimeVue + internal facade + semantic tokens; одна modern theme у MVP	Прийнято
ADR-008	Portable export окремо від operational backup	Прийнято
ADR-009	Exact storage path private by default	Прийнято
ADR-010	item_search materialized read model; bounded relevance mode	Прийнято
ADR-011	HEIC: custom libvips/libheif; public image distribution requires licensing review	Прийнято
ADR-012	Prisma CRUD + Kysely specialized Storage/Search/Jobs access; trx-aware write ports	Прийнято
ADR-013	Sync item/path/visibility projections; async vectors/import	Прийнято
ADR-014	English default/source locale; Ukrainian complete in MVP	Прийнято
ADR-015	One business transaction uses one client; item_search/outbox writes join source trx through ports	Прийнято
ADR-016	Private values не індексуються; unlisted має direct-only discoverability	Прийнято
ADR-017	Explicit item_search invalidator registry owns every denormalized source	Прийнято
ADR-018	OpenAPI єдине contract source; types і Zod генеруються разом	Прийнято
ADR-019	Bulk actions повертають per-item conflict report	Прийнято
ADR-020	ltree node label = n + lowercase UUID hex без дефісів	Прийнято

16.2 Зафіксовані MVP defaults

Область	Рішення v0.3.1
Назва	Inventory Atlas лишається робочою назвою; домен не блокує schema v1
Публічність	Public cards дозволені; exact storage path і sensitive fields private by default
Labels	A4 grid + custom 50x30 mm; thermal profile додається після вибору принтера
Коди	QR default; Code 128 optional на тому самому template
Обсяг	Acceptance dataset: 100 000 items без external search engine
Простори	Одна installation + RBAC; multi-tenant/team workspaces поза MVP
Marketplace	Manual Listing metadata у MVP; перший connector обирається окремо
Видалення	Archive first; owner-only hard delete через maintenance flow
Мови	English default; Ukrainian selectable і повна для всіх MVP flows

16.3 Наступний артефакт

Умови фінального review addendum A1-A3 і B1-B11 виконані; критичних відкритих питань для schema v1 не лишилося. Наступним документом має бути implementation blueprint: структура моногеро, database schema v1, OpenAPI skeleton, Docker Compose, CI workflow, design tokens і backlog у вигляді epics/user stories з acceptance criteria.

Додаток А. Приклад предмета

Нижче показано, як чорнова ідея перетворюється на запис у запропонованій моделі.

Поле	Значення	Джерело
display_name	Фотокамера Pentax LX - б/в	template
category	Фототехніка / Плівкові камери	core relation
type	Фотокамера	dynamic text/select
brand	Pentax	dynamic select
model	LX	dynamic text
release_year	1983	dynamic number
condition	б/в	dynamic select
storage_node	Коробка А	core relation
effective_path	Склад 1 / бок Б / коробка А	computed
visibility	public	core policy
storage_path visibility	private	node/field policy
media	1 main + N gallery	relations
code	QR URL + optional Code 128	entity_codes

Додаток В. Definition of Done для user story

- Acceptance criteria виконані й перевірені на mobile та desktop, якщо сценарій UI.
- Authorization і visibility перевірені окремими negative tests.
- Зміни API відображені в OpenAPI; зміни БД мають forward migration і rollback note.
- Frontend API client, declarations/JSDoc types і Zod runtime schemas регенеровані з однієї OpenAPI revision; checkJs проходить; codegen drift відсутній.

- Unit/integration/E2E на потрібному рівні проходять у CI.
- UI має loading, empty, validation, error, conflict і success states.
- Додані audit events для administrative або destructive actions.
- Документація deployment/configuration оновлена.
- Немає hardcoded theme colors або UI text у domain pages; English і Ukrainian locale мають coverage test без missing keys.

Додаток С. Джерела технічних рішень

Перевірено 27.08.2026. Версії dependencies мають бути зафіксовані lockfile та оновлюватися окремими pull requests.

- [1] Node.js Releases - production applications should use Active LTS або Maintenance LTS. nodejs.org/en/about/previous-releases
- [2] Vue.js FAQ - Vue 3 є поточною рекомендованою major version для нових проєктів. vuejs.org/about/faq
- [3] NestJS documentation - framework підтримує OOP, dependency injection, TypeScript та pure JavaScript. docs.nestjs.com
- [4] NestJS Fastify adapter. docs.nestjs.com/techniques/performance
- [5] NestJS OpenAPI module. docs.nestjs.com/openapi/introduction
- [6] PrimeVue styled mode - design tokens і розширення тем. primevue.org/theming/styled
- [7] PrimeVue unstyled mode - функціональність компонентів відокремлена від styling. primevue.org/theming/unstyled
- [8] PostgreSQL JSONB indexing. postgresql.org/docs/current/datatype-json.html
- [9] PostgreSQL GIN indexes and full-text search. postgresql.org/docs/current/gin.html
- [10] PostgreSQL pg_trgm similarity indexes. postgresql.org/docs/current/pgtrgm.html
- [11] Prisma JSON field support and PostgreSQL filtering. prisma.io/docs/.../working-with-json-fields
- [12] MDN BarcodeDetector - experimental and limited browser availability, secure context required. developer.mozilla.org/.../BarcodeDetector
- [13] ZXing browser - multi-format 1D/2D decoding fallback. github.com/zxing-js/browser
- [14] Нова пошта - офіційна сторінка API integration, tracking та API key. novaposhta.ua/for-business/cooperation/integration
- [15] Укрпошта - офіційна API documentation, shipment, tracking та Swagger. dev.ukrposhta.ua/documentation
- [16] PostgreSQL ltree - hierarchical path type, ancestor/descendant operators and GiST indexes. postgresql.org/docs/current/ltree.html
- [17] PostgreSQL explicit and advisory locks - transaction-level application locks. postgresql.org/docs/current/explicit-locking.html
- [18] PostgreSQL SELECT locking clause - FOR UPDATE SKIP LOCKED для queue-like tables. postgresql.org/docs/current/sql-select.html
- [19] TypeScript checkJs - static error reporting in JavaScript files. typescriptlang.org/tsconfig/#checkJs
- [20] Kysely - type-safe SQL query builder and PostgreSQL dialect. kysely.dev/docs/intro
- [21] Sharp installation/output docs - custom global libvips and HEIC/HEIF requirements. sharp.pixelplumbing.com/install;api-output/#heif

Статус документа

Final v0.3.1: review addendum A1-A3/B1-B11 та errata E1 закриті без зміни frozen scope. Усі вісім item_search invalidator paths мають атомарний projection/outbox write у source transaction. Документ затверджено як основу для schema v1 та implementation blueprint, але він не є DDL або OpenAPI YAML.